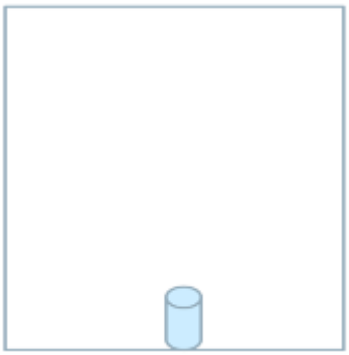
基于蒙特卡洛方法的热辐射传输模拟计算

1 引言

蒙特卡洛法（Monte Carlo Method, MCM）是基于概率统计理论的随机抽样方法，它利用重复的统计试验对所研究的物理和数学问题进行直接或间接地描述，真实反映事物特点和物理实验过程。在表面热辐射问题中，MCM 就是将表面对热辐射能的发射、吸收和反射等行为分别转化为相应的随机过程并采用相应的概率模型来模拟，每一种随机过程的概率分布复合相应的热辐射定律、表面的热辐射性质和光学原理。

计算算例(project 1)：如图所示，一房间内站着一个圆柱形的人，圆柱地面直接与地面接触，假设房间内空气不参与辐射。结构参数与表面物性自拟，并考察物性变化，采用 Monte-Carlo 方法计算：



1. 圆柱面和上端面与房间 6 个表面之间的角系数
2. 圆柱面和上端面与各个表面之间的辐射传递系数
3. 圆柱面和上端面与房间 6 个表面之间的辐射传递热量，考虑一个低于其他墙面温度的窗户的情况

2 计算过程

2.1 光子发射点的确定

将人的外形视作一个圆柱体，设其圆面半径为 R，高度为 H。分别计算端面和柱面对墙体的角系数和辐射传递系数。利用 rnd()函数生成 0 到 1 的均匀随机数。

上端圆面的发射点（x_0,y_0,H）生成方法如下：

利用随机数R_r、R_φ确定发射点的位置

$$r_0 = R_r \cdot R$$
$$\varphi_0 = R_\varphi \cdot 2\pi$$

转换为直角坐标系：

$$x_0 = r_0 \cdot cos\varphi_0$$
$$y_0 = r_0 \cdot sin\varphi_0$$

柱面发射点（x_1,y_1,z_1）生成方法如下：

利用随机数R_z、$R'_{\varphi 1}$确定发射点的位置

$$\varphi_1 = R'_{\varphi 1} \cdot 2\pi$$
$$x_1 = r_0 \cdot cos\varphi_1$$
$$y_1 = r_0 \cdot sin\varphi_1$$
$$z_1 = R_z \cdot H$$

通过随机数发生程序不断地产生随机数，就能得到均匀的光子发射点。

2.2 光子发射方向的确定

确定了光子发射点后，光子的发射方向也需要通过随机数确定。假设圆柱表面是漫射表面，光子的发射方向由两个参数决定：极角θ'和圆周角φ'，由漫射表面的发射概率模型，有下述关系：

$$\theta' = \arcsin\sqrt{R_\theta'}$$

$$\varphi' = 2\pi R_\varphi'$$

其中R_θ',R_φ'均为 0-1 之间的随机数。

2.3 墙面的吸收和反射

确定了光子发射点和发射方向，就开始跟踪光子，光子打到墙面上，就会有两种可能，被吸收或者被反射，这由墙面的吸收率（反射率）和一个随机数决定。

设墙面只吸收或反射，其吸收率为α，反射率为ρ，则有关系：

$$\alpha + \rho = 1$$

若考虑有窗户的情况，则额外考虑透射率，有关系：

$$\alpha+\rho+\tau=1$$

光子到达墙面后，产生一个随机数 R，若$R < \alpha$,则光子被吸收，反之光子被反射。

设墙面是漫反射表面。光子若被吸收，则停止对此光子的跟踪；光子若被反射，则重新产生两个随机数R_θ'',R_φ'',按 2.2 中漫射表面发射分布公式再决定被反射光子的发射角θ'', φ''，相当于进行一次墙面上的重新发射。如此循环，直至光子被某一墙面吸收。

2.4 角系数和辐射传递系数的计算

设在整个蒙特卡洛法模拟中，发射了 N 个光子，首先统计光子第一次到达的墙面（无论被吸收还是被反射），第 i 个墙面有N_i个光子首次到达，则人体对每个面的角系数为

$$X_i = \frac{N_i}{N} \qquad (i = 1,2,3,4,5,6)$$

同样，统计光子最终被第 i 个墙面吸收，每个面吸收的光子数为M_i，则人体对每个面的辐射传递系数为：

$$D_i = \frac{M_i}{N} \qquad (i = 1,2,3,4,5,6,7,8)$$

其中，角系数和辐射传的系数均满足完备性：

$$\sum_{i=1}^{6} X_i = \sum_{i=1}^{8} D_i = 1$$

3 结果与分析

针对于端面，考查了不同发射光子数目条件下的结果。
(说明：面 7 为圆柱端面，面 8 为圆柱面)
在总光子数目为 1000000 时，结果如下：

```
吸收系数取0.500000, 以人站立点为原点,
六面墙壁几何位置为:
x1=3.000000,=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
端面对面1 的角系数是0.033399
端面对面2 的角系数是0.033322
端面对面3 的角系数是0.033175
端面对面4 的角系数是0.033188
端面对面5 的角系数是0.866916
端面对面6 的角系数是0.000000
角系数之和: 1.000000
端面对面1 的辐射换热系数是0.076990
端面对面2 的辐射换热系数是0.076634
端面对面3 的辐射换热系数是0.076468
端面对面4 的辐射换热系数是0.076330
端面对面5 的辐射换热系数是0.515344
端面对面6 的辐射换热系数是0.161161
端面对面7 的辐射换热系数是0.004727
端面对面8 的辐射换热系数是0.012346
辐射换热系数之和: 1.000000
Program ended with exit code: 0
```

在总光子数目为 10000000 时，结果如下：

```
吸收系数取0.500000, 以人站立点为原点,
六面墙壁几何位置为:
x1=3.000000,=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
端面对面1 的角系数是0.033333
端面对面2 的角系数是0.033318
端面对面3 的角系数是0.033307
端面对面4 的角系数是0.033226
端面对面5 的角系数是0.866816
端面对面6 的角系数是0.000000
角系数之和: 1.000000
端面对面1 的辐射换热系数是0.076393
端面对面2 的辐射换热系数是0.076256
端面对面3 的辐射换热系数是0.076278
端面对面4 的辐射换热系数是0.076444
端面对面5 的辐射换热系数是0.516885
端面对面6 的辐射换热系数是0.160635
端面对面7 的辐射换热系数是0.004680
端面对面8 的辐射换热系数是0.012430
辐射换热系数之和: 1.000000
Program ended with exit code: 0
```

在总光子数目为 100000000 时，结果如下：

```
吸收系数取0.500000, 以人站立点为原点,
六面墙壁几何位置为:
x1=3.000000,=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
端面对面1 的角系数是0.033246
端面对面2 的角系数是0.033281
端面对面3 的角系数是0.033305
端面对面4 的角系数是0.033294
端面对面5 的角系数是0.866873
端面对面6 的角系数是0.000000
角系数之和: 1.000000
端面对面1 的辐射换热系数是0.076396
端面对面2 的辐射换热系数是0.076410
端面对面3 的辐射换热系数是0.076379
端面对面4 的辐射换热系数是0.076398
端面对面5 的辐射换热系数是0.516395
端面对面6 的辐射换热系数是0.160833
端面对面7 的辐射换热系数是0.004713
端面对面8 的辐射换热系数是0.012475
辐射换热系数之和: 1.000000
Program ended with exit code: 0
```

由于在题目假设条件下，圆柱位于房间中心，故面 1、2、3、4 应该在结果上等价。通过观察结果，1000000 个光子条件下四个等价面的辐射传递系数差距较大；而后两个情况四个等价面的辐射传递系数差距很小，如继续增加发射光子量对于结果影响不大，光子数为 100000000 的情况下得出的结果既可作为最终结果。

对于圆柱面同理，这里直接给出结果：

```
光子发射数100000000, 吸收系数取0.500000,以人站立点为原点,
六面墙壁几何位置为:
x1=3.000000,x2=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
柱面对面1 的角系数是0.123598
柱面对面2 的角系数是0.123603
柱面对面3 的角系数是0.123574
柱面对面4 的角系数是0.123609
柱面对面5 的角系数是0.165391
柱面对面6 的角系数是0.340226
角系数之和: 1.000000
柱面对面1 的辐射换热系数是0.119080
柱面对面2 的辐射换热系数是0.119089
柱面对面3 的辐射换热系数是0.119046
柱面对面4 的辐射换热系数是0.119116
柱面对面5 的辐射换热系数是0.211174
柱面对面6 的辐射换热系数是0.291041
柱面对面7 的辐射换热系数是0.000867
柱面对面8 的辐射换热系数是0.020586
辐射换热系数之和: 1.000000
Program ended with exit code: 0
```

改变表面吸收率当然会对最终结果产生相应的影响，以上结果是对于房间墙壁吸收率为 0.5 的情况下得到的。以下将讨论改变房间壁面参数下，结果的变化。

1) 令表面吸收系数为 0.01，近似于不吸收材料。因为此处为定性分析，为加速计算时间，此处减少光子发射数。圆柱面发射得到的结果为：

```
光子发射数10000000, 吸收系数取0.010000,以人站立点为原点,
六面墙壁几何位置为:
x1=3.000000,x2=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
柱面对面1 的角系数是0.123559
柱面对面2 的角系数是0.123624
柱面对面3 的角系数是0.123446
柱面对面4 的角系数是0.123822
柱面对面5 的角系数是0.165409
柱面对面6 的角系数是0.340139
角系数之和: 1.000000
柱面对面1 的辐射换热系数是0.120423
柱面对面2 的辐射换热系数是0.120136
柱面对面3 的辐射换热系数是0.120450
柱面对面4 的辐射换热系数是0.120492
柱面对面5 的辐射换热系数是0.237670
柱面对面6 的辐射换热系数是0.244905
柱面对面7 的辐射换热系数是0.001835
柱面对面8 的辐射换热系数是0.034089
辐射换热系数之和: 1.000000
Program ended with exit code: 0
```

验证： 各个面的面积比例为：18:18:18:18:36:36:0.283:3.204，
单位化后为：0.122:0.122:0.122:0.122:0.244:0.244:0.0019:0.0217；
与该情况下各个面辐射换热系数近似，考虑实际模型 z 轴方向不均匀性，结果符合实际情况。

改变各面的吸收率，各面的辐射传递系数变化情况如图 4.1 所示。

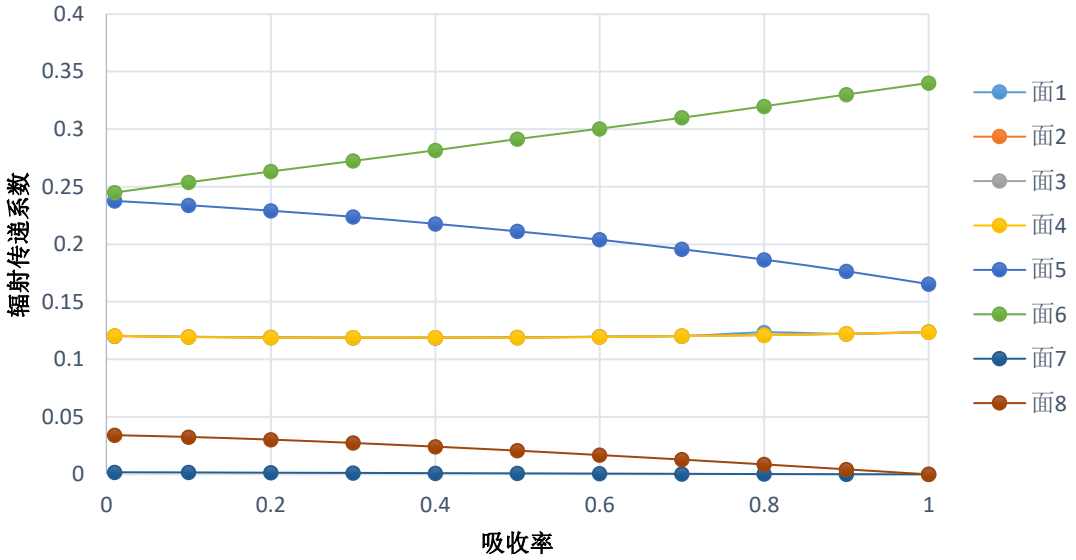


图 4.1 圆柱面对各面辐射换热系数与各面吸收率的关系

类似地，可以得到圆柱端面对各个面辐射换热系数与各面吸收率的关系，如图 4.2 所示。

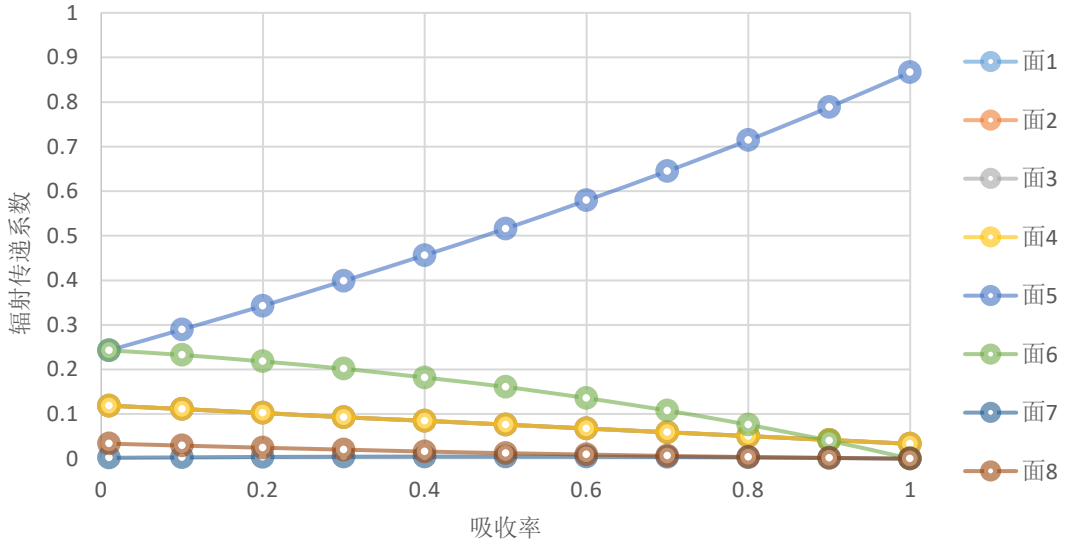


图 4.2 圆柱端面对各面辐射换热系数与各面吸收率的关系

由以上两图可知：

1) 在各面吸收率很小的情况下，各面的辐射传递系数之比约等于各面的面积比；
2) 在各面为黑体表面时，各面的辐射传递系数与其角系数相等;
3) 对于圆柱端面，随着各面吸收率的增强，顶面的辐射传递系数逐渐增强，这是由于顶面的几何位置引起的，发射出的光子有很大概率第一次会到达房间顶面。
4) 对于圆柱柱面，顶面与底面在吸收率很小的情况下十分接近，而当吸收率逐渐增大时，底面的辐射传递系数逐渐增大，顶面的辐射传递系数逐渐减小，这是由于圆柱与底面接触，发射的光子在前期与底面接触概率更大，所以被吸收的概率也更大。

4 有窗墙壁的换热情况

考虑 4＃面为有窗的墙壁，考虑其他面的吸收率为 0.5，而 4＃面由于有窗户，将其吸收率设为 0.1、反射率设为 0.2、透射率为 0.7。端面发射情况的运行结果如下：

```
吸收系数取0.500000，以人站立点为原点，
六面墙壁几何位置为：
x1=3.000000,=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
端面对面1 的角系数是0.033470
端面对面2 的角系数是0.033245
端面对面3 的角系数是0.033411
端面对面4 的角系数是0.033692
端面对面5 的角系数是0.866182
端面对面6 的角系数是0.000000
角系数之和：1.000000
面4为有窗户的墙面，吸收率为：0.100000,反射率为0.200000,透射率为0.700000
端面对面1 的辐射换热量是0.075426*Q
端面对面2 的辐射换热量是0.075294*Q
端面对面3 的辐射换热量是0.075309*Q
端面对面4 的辐射换热量是0.009170*Q
端面对面5 的辐射换热量是0.514563*Q
端面对面6 的辐射换热量是0.157277*Q
端面对面7 的辐射换热量是0.004614*Q
端面对面8 的辐射换热量是0.012119*Q
Program ended with exit code: 0
```

通过统计 4＃面吸收的光子数目，发现与其余三个等价面相比，约为它们吸收光子数目的 1/8，而 4＃面的吸收占未反射光的 1/8，模拟结果可信。

同样条件下，得到柱面对各个面的辐射换热量：

```
吸收系数取0.500000，以人站立点为原点，
六面墙壁几何位置为：
x1=3.000000,x2=-3.000000,y1=3.000000,y2=-3.000000,z1=3.000000,z2=0.000000 时
端面对面1 的角系数是0.123508
端面对面2 的角系数是0.123591
端面对面3 的角系数是0.123750
端面对面4 的角系数是0.123560
端面对面5 的角系数是0.165387
端面对面6 的角系数是0.340205
角系数之和：1.000000
面4为有窗户的墙面，吸收率为：0.100000,反射率为0.200000,透射率为0.700000
端面对面1 的辐射换热量是0.114279*Q
端面对面2 的辐射换热量是0.114439*Q
端面对面3 的辐射换热量是0.115622*Q
端面对面4 的辐射换热量是0.017729*Q
端面对面5 的辐射换热量是0.202197*Q
端面对面6 的辐射换热量是0.281426*Q
端面对面7 的辐射换热量是0.000818*Q
端面对面8 的辐射换热量是0.019472*Q
Program ended with exit code: 0
```

5 源代码

无窗情况：

```
/*端面发射*/
#include<stdlib.h>
#include<stdio.h>
#include<time.h>
#include<math.h>
#include<iostream>
#define N 10000000
#define pi 3.141592653589793
#define R 0.3
#define Xn -3.0 //  定义全局常量  X_negative X_positive...
#define Xp 3.0
#define Yn -3.0
#define Yp 3.0
#define Zn 0.0
#define Zp 3.0
#define H    1.7
#define ABSORB 0.5
#define min(x,y)    ( x<y?x:y )//取小
using namespace std;

/*定义结构体表示点的坐标以及在哪个面上*/
typedef struct facepoint
{int num;double x;double y;double z;} FACEPOINT;
/*定义发射方向*/
typedef struct direction
{double x;double y;double z;} DRCT;
/*生成0到1范围的随机数*/
double rnd()
{
      double j ;
      j = (double)    rand ( ) / (RAND_MAX);
      return j;
}
int temp;
/*构造求交点的函数（第二次及以后）*/
FACEPOINT nextPoint (DRCT a , FACEPOINT d ){
      double t[9];    int num=0;double mint = 0.0;
      t[0]=( Xp-d.x)/a.x;//到六个壁面线段距离
      t[1]=( Xn-d.x)/a.x;
      t[2]=( Yp-d.y)/a.y;
      t[3]=( Yn-d.y)/a.y;
      t[4]=( Zp-d.z)/a.z;
      t[5]=( Zn-d.z)/a.z;
      if(pow(d.x+t[5]*a.x,2)+pow(d.y+t[5]*a.y,2)<R*R)//排除遮挡面
            t[5]= -1;

      t[6] =(H - d.z)/a.z;
      //判断与端面是否有交点
      if ((d.x+t[6]*a.x)*(d.x+t[6]*a.x)+(d.y+t[6]*a.y)*(d.y+t[6]*a.y)>R*R)
            t[6]= -1;
```

```
	//判断与柱面是否有交点
	double A = (a.x*a.x+a.y*a.y);
	double B = 2*(a.x*d.x+a.y*d.y);
	double C = d.x*d.x + d.y*d.y - R*R;
	if (delta < 0)    t[7] = t[8]= -1;
	else{
		t[7] = (- B+ sqrt(delta))/(2*A);
		t[8] = (- B - sqrt(delta))/(2*A);
	 //排除延伸面
		if (d.z + t[7]* a.z > H || d.z + t[7]* a.z<0 ) t[7] = -1;
		if (d.z + t[8]* a.z > H || d.z + t[8]* a.z<0 ) t[8] = -1;

	}
	//将负数时间赋予大值1023
	for(int i = 0; i<9; i++){
		if (t[i] <= 0) t[i] = 1023;
	}
	//找出最小的时间
	mint=t[0];

	for(int i=0;i<9;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<mint){
			mint=t[i];
			num=i;
		}
	if (num==8) {
		num=7;
	}
	//求出交点坐标
	double x1= d.x+mint*a.x, y1 = d.y + mint* a.y ,z1 = d.z + mint * a.z;
	FACEPOINT np;
	np.x= x1; np.y = y1; np.z = z1; np.num = num;
	return np;
}
/*构造求初次交点的函数*/
FACEPOINT firstpoint(double x1, double y1, double    z1, double x2,double y2, double z2)
{
	double t[6]; int k=0,    i=0 ; int num=0;
	t[0]=( Xp-x1 )/x2;//求线段距离
	t[1]=( Xn-x1 )/x2;
	t[2]=( Yp-y1 )/y2;
	t[3]=( Yn-y1 )/y2;
	t[4]=( Zp-z1 )/z2;
	t[5]=( Zn-z1 )/z2;

	for(k=0;k<6;k++)//将距离为负数的赋值为一个足够大的值1023，不会是最小值即可
		if(t[k]<0){
			t[k]=1023;
		}
	double mint=t[0];
	for(i=0;i<6;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<mint){
			mint=t[i];
			num=i;
		}
```

```
    FACEPOINT d;
    d.num=num;
    d.x=x1+t[num]*x2;
    d.y=y1+t[num]*y2;
    d.z=z1+t[num]*z2;
    return d;//返回坐标和相交平面
}
/*构造求发射方向的函数*/
DRCT directExch(FACEPOINT p){
    double the2 =0, phi2 = 0,   x2 = 0,   y2 = 0,   z2 = 0;
    the2 = asin(sqrt(rnd()));
    phi2 = 2*pi*rnd();
    z2 = cos(the2);
    x2 = sin(the2)*cos(phi2);
    y2 = sin(the2)*sin(phi2);

    DRCT d;
    switch(p.num)//将前一个交点的面的标号带入，坐标变换
    {
        case 0: d.x = -z2; d.y = y2; d.z = x2; break;
        case 1: d.x =   z2; d.y = y2; d.z = x2; break;
        case 2: d.y = -z2; d.x = x2; d.z = y2; break;
        case 3: d.y =   z2; d.x = x2; d.z = y2; break;
        case 4: d.z = -z2; d.x = x2; d.y = y2; break;
        case 5: d.z =   z2; d.x = x2; d.y = y2; break;
        case 6: d.z =   z2; d.x = x2; d.y = y2; break;
        case 7: d.z =   x2; d.x = y2*p.y/R+z2*p.x/R;
            d.y = -y2*p.x/R+z2*p.y/R;          break;
    }
    return d;
}

int main()
{
    double raf_num[6]={0},rdi_num[9]={0},sum=0;//对六个面的相交记录,对八个面的吸收记录。
    time_t t;
    srand((unsigned) time(&t)*10);//设定seed
    for(int k=0;k<N;k++)
    {

        //端面
        double x1,y1,z1,r0,ang0;//发射点位置
        double the2,phi2,x2,y2,z2;//发射方向

        z1=H;
        while(true){//生成端面内的发射点
            r0=rnd()*R;
            ang0=2*pi*rnd();
            x1=r0*cos(ang0);
            y1=r0*sin(ang0);

            break;
        }
        /*确定发射方向*/
        the2 = asin(sqrt(rnd()));
        phi2 = 2.0*pi*rnd();
```

```
            z2 = cos(the2);
            x2 = sin(the2)*cos(phi2);
            y2 = sin(the2)*sin(phi2);
            /*第一个交点*/
            FACEPOINT point1;
            point1 = firstpoint(x1,y1,z1,x2,y2,z2);//返回发射点位置与角度相关值
            raf_num[point1.num]++; //记录角系数
            /*多次反射交点*/
            for( ;rnd()>ABSORB; )//如果没有被吸收
            {
                DRCT a;
                a = directExch(point1);
                point1 = nextPoint(a,point1);
            }
            rdi_num[point1.num]++;//记录R系数
        }
        printf("吸收系数取%f，以人站立点为原点,\n六面墙壁几何位置
为:\nx1=%f,\x2=%f,y1=%f,y2=%f,z1=%f,z2=%f  时\n", ABSORB,Xp,Xn,Yp,Yn,Zp,Zn);
        double raf[6],rdi[9];
        for(int j=0;j<6;j++){
            raf[j] = raf_num[j]/N;
            printf("端面对面%d  的角系数是%f\n", j +1 , raf[j]);
            sum=sum+raf[j];

        }
        printf("角系数之和：%f\n",sum);
        sum=0;
        rdi[7]=rdi[8]+rdi[7];
        for(int i = 0; i < 8 ; i++){
            rdi[i] = rdi_num[i]/N;
            printf("端面对面%d  的辐射换热系数是%f\n", i +1 , rdi[i]);
              sum=sum+rdi[i];
        }
        printf("辐射换热系数之和：%f\n",sum);
        return 0;
        }
```

```
//
//  main.cpp
//  zhumian
//
//  Created by 黄天成 on 16/5/20.
//  Copyright © 2016年 黄天成. All rights reserved.
//

/*柱面发射*/
#include<stdlib.h>
#include<stdio.h>
#include<time.h>
#include<math.h>
#include<iostream>
#define N 100000000
#define pi 3.141592653589793
#define R 0.3
#define Xn -3.0// 定义全局常量 X_negative X_positive...
#define Xp 3.0
#define Yn -3.0
#define Yp 3.0
#define Zn 0.0
#define Zp 3.0
#define H  1.7
#define ABSORB 0.5

using namespace std;
//定义数据类型
/*定义结构体表示点的坐标以及在哪个面上*/
typedef struct facepoint
{int num;double x;double y;double z;} FACEPOINT;
/*定义发射方向*/
typedef struct direction
{double x;double y;double z;} DRCT;
//////////////////////////////////////////////////////////定义函数
/*生成0到1范围的随机数 —— 无输入 —— 输出随机数double j */
double rnd()
{
    double j ;
    j = (double)  rand ( ) / (RAND_MAX);
    return j;
}
/*构造求交点的函数 —— 输入FACEPOINT p1 和 DRCT d1 —— 输出FACEPOINT p2*/
FACEPOINT nextPoint (DRCT a , FACEPOINT d )
{
    double t[9]; int num=0;double mint = 0.0;
    t[0]=( Xp-d.x)/a.x;
    t[1]=( Xn-d.x)/a.x;
    t[2]=( Yp-d.y)/a.y;
    t[3]=( Yn-d.y)/a.y;
    t[4]=( Zp-d.z)/a.z;
```

```
	t[5]=( Zn-d.z)/a.z;
	if(pow(d.x+t[5]*a.x,2)+pow(d.y+t[5]*a.y,2)<R*R)//排除遮挡面
		t[5]= -1;

	t[6] =(H - d.z)/a.z;
	if ((d.x+t[6]*a.x)*(d.x+t[6]*a.x)+(d.y+t[6]*a.y)*(d.y+t[6]*a.y)>R*R)
		t[6]= -1;
	//判断与柱面是否有交点
	double A = (a.x*a.x+a.y*a.y);
	double B = 2*(a.x*d.x+a.y*d.y);
	double C = d.x*d.x + d.y*d.y - R*R;double delta = B*B - 4 * A * C;
	if (delta < 0)    t[7] = t[8]= -1;
	else{
		t[7] = (- B+ sqrt(delta))/(2*A);
		t[8] = (- B - sqrt(delta))/(2*A);
		if (d.z + t[7]* a.z > H || d.z + t[7]* a.z<0 ) t[7] = -1;
		if (d.z + t[8]* a.z > H || d.z + t[8]* a.z<0 ) t[8] = -1;

	}
	//将负数时间赋予大值
	for(int i = 0; i<9; i++){
		if (t[i] <= 0) t[i] = 1023;
	}
	//找出最小的时间
	mint=t[0];

	for(int i=0;i<9;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<=mint){
			mint=t[i];
			num=i;
		}
	if (num==8) {
		num=7;
	}
	//求出交点坐标

	double x1= d.x + mint * a.x, y1 = d.y + mint * a.y ,z1 = d.z + mint * a.z;
	FACEPOINT np;
	np.x= x1; np.y = y1; np.z = z1; np.num = num;
	return np;
}
/*构造求初次交点的函数 —— 输入FACEPOINT p1 和 DRCT d1 —— 输出FACEPOINT p2 */
FACEPOINT firstpoint(FACEPOINT p1, DRCT d1 )
{
	double t[6]; int k=0,    i=0 ; int num=0;
	t[0]=( Xp-p1.x )/d1.x;
	t[1]=( Xn-p1.x )/d1.x;
	t[2]=( Yp-p1.y )/d1.y;
	t[3]=( Yn-p1.y )/d1.y;
	t[4]=( Zp-p1.z )/d1.z;
	t[5]=( Zn-p1.z )/d1.z;

	for(k=0;k<6;k++)//将时间为负数的赋值为一个足够大的值1023
		if(t[k]<=0){
			t[k]=1023;
		}
```

```
	double mint=t[0];
	for(i=0;i<6;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<=mint){
			mint=t[i];
			num=i;
		}

	FACEPOINT p2;
	p2.num=num;
	p2.x=p1.x+mint*d1.x;
	p2.y=p1.y+mint*d1.y;
	p2.z=p1.z+mint*d1.z;
	return p2;
}
/*构造求发射方向的函数 —— 输入FACEPOINT p —— 输出 DRCT d*/
DRCT directExch(FACEPOINT p){
	double the2 =0, phi2 = 0,   x2 = 0,   y2 = 0,   z2 = 0;
	the2 = asin(sqrt(rnd()));

	phi2 = 2*pi*rnd();
	z2 = cos(the2);
	x2 = sin(the2)*cos(phi2);
	y2 = sin(the2)*sin(phi2);

	DRCT d;
	switch(p.num)
	{
		case 0: d.x = -z2; d.y = y2; d.z = x2; break;
		case 1: d.x =   z2; d.y = y2; d.z = x2; break;
		case 2: d.y = -z2; d.x = x2; d.z = y2; break;
		case 3: d.y =   z2; d.x = x2; d.z = y2; break;
		case 4: d.z = -z2; d.x = x2; d.y = y2; break;
		case 5: d.z =   z2; d.x = x2; d.y = y2; break;
		case 6: d.z =   z2; d.x = x2; d.y = y2; break;
		case 7: d.z =   x2; d.x = y2*p.y/R+z2*p.x/R;
			d.y = -y2*p.x/R+z2*p.y/R;          break;
	}
	return d;
}

//主函数
int main()
{
	double raf_num[6]={0},rdi_num[8]={0},sum=0; //对六个面的相交记录,对八个面的吸收记录。
	time_t t;
	srand((unsigned) time(&t)*10);//设定seed
	for(int k=0;k<N;k++)
	{
		//柱面
		double x1, y1,z1;//发射点位置
		double the2,phi2,x2,y2,z2,x3,y3,z3,phi1;//发射方向
		/*生成柱面内的发射点*/
		z1 = H*rnd();
		phi1 = 2*pi*rnd();
		x1 = R*cos(phi1);
		y1 = R*sin(phi1);
```

```
            FACEPOINT p1;
            p1.x=x1; p1.y=y1; p1.z=z1; p1.num=0;
            /*确定发射方向*/
            the2 = asin(sqrt(rnd()));
            phi2 = 2*pi*rnd();
            z3 = cos(the2);
            x3 = sin(the2)*cos(phi2);
            y3 = sin(the2)*sin(phi2);
            /*坐标转换*/
            z2 = x3;
            x2 = y3*sin(phi1)+z3*cos(phi1);
            y2 = -y3*cos(phi1)+z3*sin(phi1);
            DRCT d1;
            d1.x=x2;d1.y=y2;d1.z=z2;
            /*第一个交点*/
            FACEPOINT point1 = firstpoint(p1,d1);
            raf_num[point1.num]++;
            /*多次反射交点*/
            FACEPOINT point2=point1;
            for( ;rnd()>ABSORB; )
            {
                DRCT a= directExch(point2);
                point2 = nextPoint(a,point2);

            }
            rdi_num[point2.num]++;
        }
        printf("光子发射数%d, 吸收系数取%f,以人站立点为原点,\n六面墙壁几何位置
为:\nx1=%f,x2=%f,y1=%f,y2=%f,z1=%f,z2=%f 时\n",N, ABSORB,Xp,Xn,Yp,Yn,Zp,Zn);
        double raf[6]={0},rdi[8]={0};
        for(int j=0;j<6;j++)
        {
            raf[j] = raf_num[j]/N;
            printf("柱面对面%d 的角系数是%f\n", j +1 , raf[j]);
            sum=sum+raf[j];
        }
        printf("角系数之和：%f\n",sum);
        sum=0;
        for(int i = 0; i < 8 ; i++)
        {
            rdi[i] = rdi_num[i]/N;
            printf("柱面对面%d 的辐射换热系数是%f\n", i +1 , rdi[i]);
            sum=sum+rdi[i];
        }
        printf("辐射换热系数之和：%f\n",sum);

        return 0;
}
```

有窗情况：

```
/*端面发射*/
#include<stdlib.h>
#include<stdio.h>
#include<time.h>
#include<math.h>
#include<iostream>
#define N 1000000
#define pi 3.141592653589793
#define R 0.3
#define Xn -3.0 //  定义全局常量  X_negative X_positive...
#define Xp 3.0
#define Yn -3.0
#define Yp 3.0
#define Zn 0.0
#define Zp 3.0
#define H    1.7
#define ABSORB 0.5
#define ABSORB1 0.1
#define REFLEC 0.2
#define min(x,y)    ( x<y?x:y )//取小
using namespace std;

/*定义结构体表示点的坐标以及在哪个面上*/
typedef struct facepoint
{int num;double x;double y;double z;} FACEPOINT;
/*定义发射方向*/
typedef struct direction
{double x;double y;double z;} DRCT;
/*生成0到1范围的随机数*/
double rnd()
{
      double j ;
      j = (double)    rand ( ) / (RAND_MAX);
      return j;
}
int temp;
/*构造求交点的函数（第二次及以后）*/
FACEPOINT nextPoint (DRCT a , FACEPOINT d ){
      double t[9];    int num=0;double mint = 0.0;
      t[0]=( Xp-d.x)/a.x;//到六个壁面线段距离
      t[1]=( Xn-d.x)/a.x;
      t[2]=( Yp-d.y)/a.y;
      t[3]=( Yn-d.y)/a.y;
      t[4]=( Zp-d.z)/a.z;
      t[5]=( Zn-d.z)/a.z;
      if(pow(d.x+t[5]*a.x,2)+pow(d.y+t[5]*a.y,2)<R*R)//排除遮挡面
            t[5]= -1;

      t[6] =(H - d.z)/a.z;
      //判断与端面是否有交点
      if ((d.x+t[6]*a.x)*(d.x+t[6]*a.x)+(d.y+t[6]*a.y)*(d.y+t[6]*a.y)>R*R)
            t[6]= -1;
      //判断与柱面是否有交点
```

```
	double A = (a.x*a.x+a.y*a.y);
	double B = 2*(a.x*d.x+a.y*d.y);
	double C = d.x*d.x + d.y*d.y - R*R;
	double delta = B*B - 4 * A * C;
	if (delta < 0)    t[7] = t[8]= -1;
	else{
		t[7] = (- B+ sqrt(delta))/(2*A);
		t[8] = (- B - sqrt(delta))/(2*A);
		if (d.z + t[7]* a.z > H || d.z + t[7]* a.z<0 ) t[7] = -1;//排除延伸面
		if (d.z + t[8]* a.z > H || d.z + t[8]* a.z<0 ) t[8] = -1;

	}
	//将负数时间赋予大值1023
	for(int i = 0; i<9; i++){
		if (t[i] <= 0) t[i] = 1023;
	}
	//找出最小的时间
	mint=t[0];

	for(int i=0;i<9;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<mint){
			mint=t[i];
			num=i;
		}
	if (num==8) {
		num=7;
	}
	//求出交点坐标
	double x1= d.x+mint*a.x, y1 = d.y + mint* a.y ,z1 = d.z + mint * a.z;
	FACEPOINT np;
	np.x= x1; np.y = y1; np.z = z1; np.num = num;
	return np;
}
/*构造求初次交点的函数*/
FACEPOINT firstpoint(double x1, double y1, double    z1, double x2,double y2, double z2)
{
	double t[6]; int k=0,    i=0 ; int num=0;
	t[0]=( Xp-x1 )/x2;//求线段距离
	t[1]=( Xn-x1 )/x2;
	t[2]=( Yp-y1 )/y2;
	t[3]=( Yn-y1 )/y2;
	t[4]=( Zp-z1 )/z2;
	t[5]=( Zn-z1 )/z2;

	for(k=0;k<6;k++)//将距离为负数的赋值为一个足够大的值1023，不会是最小值即可
		if(t[k]<0){
			t[k]=1023;
		}
	double mint=t[0];
	for(i=0;i<6;i++)//找出数组中的最小值，对应编号即为相交平面编号。
		if(t[i]<mint){
			mint=t[i];
			num=i;
		}

	FACEPOINT d;
```

```
	d.num=num;
	d.x=x1+t[num]*x2;
	d.y=y1+t[num]*y2;
	d.z=z1+t[num]*z2;
	return d;//返回坐标和相交平面
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/*构造求发射方向的函数*/
DRCT directExch(FACEPOINT p){
	double the2 =0, phi2 = 0,   x2 = 0,   y2 = 0,   z2 = 0;
	the2 = asin(sqrt(rnd()));
	phi2 = 2*pi*rnd();
	z2 = cos(the2);
	x2 = sin(the2)*cos(phi2);
	y2 = sin(the2)*sin(phi2);

	DRCT d;
	switch(p.num)//将前一个交点的面的标号带入，坐标变换
	{
		case 0: d.x = -z2; d.y = y2; d.z = x2; break;
		case 1: d.x =    z2; d.y = y2; d.z = x2; break;
		case 2: d.y = -z2; d.x = x2; d.z = y2; break;
		case 3: d.y =    z2; d.x = x2; d.z = y2; break;
		case 4: d.z = -z2; d.x = x2; d.y = y2; break;
		case 5: d.z =    z2; d.x = x2; d.y = y2; break;
		case 6: d.z =    z2; d.x = x2; d.y = y2; break;
		case 7: d.z =    x2; d.x = y2*p.y/R+z2*p.x/R;
			d.y = -y2*p.x/R+z2*p.y/R;          break;
	}
	return d;
}

int main()
{
	double raf_num[6]={0},rdi_num[9]={0},sum=0;//对六个面的相交记录,对八个面的吸收记录。
	time_t t;
	srand((unsigned) time(&t)*10);//设定seed
	for(int k=0;k<N;k++)
	{

		//端面
		double x1,y1,z1,r0,ang0;//发射点位置
		double the2,phi2,x2,y2,z2;//发射方向

		z1=H;
		while(true){//生成端面内的发射点
			r0=rnd()*R;
			ang0=2*pi*rnd();
			x1=r0*cos(ang0);
			y1=r0*sin(ang0);

			break;
		}
		/*确定发射方向*/

		the2 = asin(sqrt(rnd()));
```

```
            phi2 = 2.0*pi*rnd();
            z2 = cos(the2);
            x2 = sin(the2)*cos(phi2);
            y2 = sin(the2)*sin(phi2);
            /*第一个交点*/
            FACEPOINT point1;
            point1 = firstpoint(x1,y1,z1,x2,y2,z2);//返回发射点位置与角度相关值
            raf_num[point1.num]++; //记录角系数
            /*多次反射交点*/
            double flag=rnd();
            if (point1.num==3) {
                for( ;flag>(1-REFLEC); )//如果没有被吸收
                {
                    DRCT a;
                    a = directExch(point1);
                    point1 = nextPoint(a,point1);
                    flag=rnd();
                }

            }
            else
                for( ;rnd()>ABSORB; )//如果没有被吸收
            {
                DRCT a;
                a = directExch(point1);
                point1 = nextPoint(a,point1);
            }

            if (point1.num==3) {
                if (flag<ABSORB1) {
                    rdi_num[point1.num]++;

                }
            }
            else rdi_num[point1.num]++;//记录R系数
        }
        printf("吸收系数取%f，以人站立点为原点,\n六面墙壁几何位置
为:\nx1=%f,\x2=%f,y1=%f,y2=%f,z1=%f,z2=%f  时\n", ABSORB,Xp,Xn,Yp,Yn,Zp,Zn);
        double raf[6],rdi[9];
        for(int j=0;j<6;j++){
            raf[j] = raf_num[j]/N;
            printf("端面对面%d  的角系数是%f\n", j +1 , raf[j]);
            sum=sum+raf[j];

        }
        printf("角系数之和：%f\n",sum);
        printf("面4为有窗户的墙面，吸收率为：%f,反射率为%f,透射率为%f\n",ABSORB1,REFLEC,(1-
ABSORB1-REFLEC));
        sum=0;
        rdi[7]=rdi[8]+rdi[7];
        for(int i = 0; i < 8 ; i++){
            rdi[i] = rdi_num[i]/N;
            printf("端面对面%d  的辐射换热量是%f＊Q\n", i +1 , rdi[i]);
            sum=sum+rdi[i];
        }
```

```
	//printf("辐射换热系数之和：%f\n",sum);
	return 0;

}



/*柱面发射*/
#include<stdlib.h>
#include<stdio.h>
#include<time.h>
#include<math.h>
#include<iostream>
#define N 10000000
#define pi 3.141592653589793
#define R 0.3
#define Xn -3.0//  定义全局常量  X_negative X_positive...
#define Xp 3.0
#define Yn -3.0
#define Yp 3.0
#define Zn 0.0
#define Zp 3.0
#define H    1.7
#define ABSORB 0.5
#define ABSORB1 0.1
#define REFLEC 0.2


using namespace std;
//////////////////////////////////////////////////////////////////定义数据类型
/*定义结构体表示点的坐标以及在哪个面上*/
typedef struct facepoint
{int num;double x;double y;double z;} FACEPOINT;
/*定义发射方向*/
typedef struct direction
{double x;double y;double z;} DRCT;
//////////////////////////////////////////////////////////////////定义函数
/*生成0到1范围的随机数 —— 无输入 —— 输出随机数double j */
double rnd()
{
	double j ;
	j = (double)    rand ( ) / (RAND_MAX);
	return j;
}
/*构造求交点的函数 —— 输入FACEPOINT p1  和  DRCT d1 —— 输出FACEPOINT p2*/
FACEPOINT nextPoint (DRCT a , FACEPOINT d )
{
	double t[9]; int num=0;double mint = 0.0;
	t[0]=( Xp-d.x)/a.x;
	t[1]=( Xn-d.x)/a.x;
	t[2]=( Yp-d.y)/a.y;
	t[3]=( Yn-d.y)/a.y;
	t[4]=( Zp-d.z)/a.z;
	t[5]=( Zn-d.z)/a.z;
	if(pow(d.x+t[5]*a.x,2)+pow(d.y+t[5]*a.y,2)<R*R)//排除遮挡面
```

```
        t[5]= -1;

    t[6] =(H - d.z)/a.z;
    if ((d.x+t[6]*a.x)*(d.x+t[6]*a.x)+(d.y+t[6]*a.y)*(d.y+t[6]*a.y)>R*R)
        t[6]= -1;
    //判断与柱面是否有交点
    double A = (a.x*a.x+a.y*a.y);
    double B = 2*(a.x*d.x+a.y*d.y);
    double C = d.x*d.x + d.y*d.y - R*R;
    double delta = B*B - 4 * A * C;
    if (delta < 0)    t[7] = t[8]= -1;
    else{
        t[7] = (- B+ sqrt(delta))/(2*A);
        t[8] = (- B - sqrt(delta))/(2*A);
        if (d.z + t[7]* a.z > H || d.z + t[7]* a.z<0 ) t[7] = -1;//排除延伸面
        if (d.z + t[8]* a.z > H || d.z + t[8]* a.z<0 ) t[8] = -1;

    }
    //将负数时间赋予大值
    for(int i = 0; i<9; i++){
        if (t[i] <= 0) t[i] = 1023;
    }
    //找出最小的时间
    mint=t[0];

    for(int i=0;i<9;i++)//找出数组中的最小值，对应编号即为相交平面编号。
        if(t[i]<=mint){
            mint=t[i];
            num=i;
        }
    if (num==8) {
        num=7;
    }
    //求出交点坐标

    double x1= d.x + mint * a.x, y1 = d.y + mint * a.y ,z1 = d.z + mint * a.z;
    FACEPOINT np;
    np.x= x1; np.y = y1; np.z = z1; np.num = num;
    return np;
}
/*构造求初次交点的函数 —— 输入FACEPOINT p1 和 DRCT d1 —— 输出FACEPOINT p2 */
FACEPOINT firstpoint(FACEPOINT p1, DRCT d1 )
{
    double t[6]; int k=0,    i=0 ; int num=0;
    t[0]=( Xp-p1.x )/d1.x;
    t[1]=( Xn-p1.x )/d1.x;
    t[2]=( Yp-p1.y )/d1.y;
    t[3]=( Yn-p1.y )/d1.y;
    t[4]=( Zp-p1.z )/d1.z;
    t[5]=( Zn-p1.z )/d1.z;

    for(k=0;k<6;k++)//将时间为负数的赋值为一个足够大的值1023
        if(t[k]<=0){
            t[k]=1023;
        }
    double mint=t[0];
```

```
    for(i=0;i<6;i++)//找出数组中的最小值，对应编号即为相交平面编号。
        if(t[i]<=mint){
            mint=t[i];
            num=i;
        }

    FACEPOINT p2;
    p2.num=num;
    p2.x=p1.x+mint*d1.x;
    p2.y=p1.y+mint*d1.y;
    p2.z=p1.z+mint*d1.z;
    return p2;
}
/*构造求发射方向的函数 —— 输入FACEPOINT p —— 输出 DRCT d*/
DRCT directExch(FACEPOINT p){
    double the2 =0, phi2 = 0,   x2 = 0,   y2 = 0,   z2 = 0;
    the2 = asin(sqrt(rnd()));

    phi2 = 2*pi*rnd();
    z2 = cos(the2);
    x2 = sin(the2)*cos(phi2);
    y2 = sin(the2)*sin(phi2);

    DRCT d;
    switch(p.num)
    {
        case 0: d.x = -z2; d.y = y2; d.z = x2; break;
        case 1: d.x =   z2; d.y = y2; d.z = x2; break;
        case 2: d.y = -z2; d.x = x2; d.z = y2; break;
        case 3: d.y =   z2; d.x = x2; d.z = y2; break;
        case 4: d.z = -z2; d.x = x2; d.y = y2; break;
        case 5: d.z =   z2; d.x = x2; d.y = y2; break;
        case 6: d.z =   z2; d.x = x2; d.y = y2; break;
        case 7: d.z =   x2; d.x = y2*p.y/R+z2*p.x/R;
            d.y = -y2*p.x/R+z2*p.y/R;          break;
    }
    return d;
}
////////////////////////////////////////////////////////////////////////主函数
int main()
{
    double raf_num[6]={0},rdi_num[8]={0},sum=0; //对六个面的相交记录,对八个面的吸收记录。
    time_t t;
    srand((unsigned) time(&t)*10);//设定seed
    for(int k=0;k<N;k++)
    {
        //柱面
        double x1, y1,z1;//发射点位置
        double the2,phi2,x2,y2,z2,x3,y3,z3,phi1;//发射方向
        /*生成柱面内的发射点*/
        z1 = H*rnd();
        phi1 = 2*pi*rnd();
        x1 = R*cos(phi1);
        y1 = R*sin(phi1);
        FACEPOINT p1;
        p1.x=x1; p1.y=y1; p1.z=z1; p1.num=0;
```

```
            /*确定发射方向*/
            the2 = asin(sqrt(rnd()));
            phi2 = 2*pi*rnd();
            z3 = cos(the2);
            x3 = sin(the2)*cos(phi2);
            y3 = sin(the2)*sin(phi2);
            /*坐标转换*/
            z2 = x3;
            x2 = y3*sin(phi1)+z3*cos(phi1);
            y2 = -y3*cos(phi1)+z3*sin(phi1);
            DRCT d1;
            d1.x=x2;d1.y=y2;d1.z=z2;
            /*第一个交点*/
            FACEPOINT point1 = firstpoint(p1,d1);
            raf_num[point1.num]++;
            /*多次反射交点*/
            double flag=rnd();
            if (point1.num==3) {
                for( ;flag>(1-REFLEC); )//如果没有被吸收
                {
                    DRCT a;
                    a = directExch(point1);
                    point1 = nextPoint(a,point1);
                    flag=rnd();
                }

            }
            else
                for( ;rnd()>ABSORB; )//如果没有被吸收
                {
                    DRCT a;
                    a = directExch(point1);
                    point1 = nextPoint(a,point1);
                }

            if (point1.num==3) {
                if (flag<ABSORB1) {
                    rdi_num[point1.num]++;

                }
            }
            else rdi_num[point1.num]++;//记录R系数
        }
        printf("吸收系数取%f，以人站立点为原点,\n六面墙壁几何位置
为:\nx1=%f,x2=%f,y1=%f,y2=%f,z1=%f,z2=%f  时\n", ABSORB,Xp,Xn,Yp,Yn,Zp,Zn);
        double raf[6],rdi[9];
        for(int j=0;j<6;j++){
            raf[j] = raf_num[j]/N;
            printf("端面对面%d  的角系数是%f\n", j +1 , raf[j]);
            sum=sum+raf[j];

        }
        printf("角系数之和：%f\n",sum);
        printf("面4为有窗户的墙面，吸收率为：%f,反射率为%f,透射率为%f\n",ABSORB1,REFLEC,(1-
ABSORB1-REFLEC));
```

```
	sum=0;
	rdi[7]=rdi[8]+rdi[7];
	for(int i = 0; i < 8 ; i++){
		rdi[i] = rdi_num[i]/N;
		printf("端面对面%d  的辐射换热量是%f＊Q\n", i +1 , rdi[i]);
		sum=sum+rdi[i];
	}
	//printf("辐射换热系数之和：%f\n",sum);
	return 0;

}
```